

Khulna University of Engineering & Technology

Department of Computer Science and Engineering

PROJECT PROGRESS REPORT

Sentinel

A Procedurally Generated, Hierarchically Animated
Night-Time Forward Operating Base Renderer

Md. Shifat Hasan (2107067)

Course: Computer Graphics and Image Processing Laboratory (CSE 4102)
Year / Term: 4th Year, 1st Term
Lab Group: B1
Instructors: Md. Tajmilur Rahman
Md. Mubtashim Abrar Nihal

September 23, 2026

Contents

1	Introduction	2
1.1	Objectives	2
1.2	Scope of this report	3
2	Technology Stack and Development Environment	3
3	Design Constraints and Principles	3
4	System Architecture	4
5	Procedural Geometry Generation	5
5.1	Base primitives	5
5.2	Composition via runtime transforms	5
6	Scene Composition	5
7	Transform Hierarchy System	7
8	Camera System and Projections	7
9	Operating Modes	8
9.1	Patrol Mode (automatic)	8
9.2	Inspection Mode (interactive)	8
10	Verification and Testing Methodology	8
11	Current Progress Status	9
12	Remaining Work	10
13	Conclusion	10

Abstract

SENTINEL is a real-time, hand-built renderer of a small night-time Forward Operating Base (FOB), written from scratch in the Odin programming language on top of the OpenGL core profile, hand-written GLSL, and GLFW, with no game engine, no scene-graph library, and no imported models, textures, or precomputed geometry of any kind. This report documents progress as of the current milestone: a custom, theme-agnostic OpenGL abstraction layer; a procedural geometry system restricted to three base primitives (cube, tetrahedron, plane), from which every other shape in the scene – cylinders, cones, wedges, and full composite objects – is assembled at runtime by combining primitive instances through matrix transforms; a hand-rolled parent/local-transform scene hierarchy supporting nested articulation; nine procedurally generated scene objects, several with multi-level hierarchical children; a free-fly camera with live-switchable perspective and orthographic projections; an automatic *Patrol Mode* that animates the scene through pure time-driven formulas; and an interactive *Inspection Mode* that supports object selection (by keyboard cycling and real mouse-ray picking), and live translate/rotate editing of any selected object's transform. All geometry, object placement, and animation is generated by runtime formulas – nothing is hardcoded as a literal table of numbers, and nothing is cached as though it were static data. The project is organized into independently testable Odin packages, with 49 automated unit tests currently passing and a headless screenshot-capture mechanism used throughout development for self-verification. Illumination, shading, culling/hidden-surface-removal demonstrations, and the ray/path/ photon-tracing components of the course syllabus are planned for the next development phase and are intentionally out of scope for this report.

1 Introduction

Modern real-time computer graphics rests on a small set of recurring ideas – procedural construction of geometry, hierarchical transformation of that geometry, camera and projection mathematics, and, ultimately, illumination – that are far better understood by building a renderer from first principles than by treating a graphics engine as a black box. **SENTINEL** is a course project built on exactly that premise: a small, self-contained, night-time military base scene, rendered by a renderer whose every mesh, object placement, and animation is produced by code executed at runtime, never authored as a static asset or a literal data table.

The *Forward Operating Base* theme was chosen deliberately. A small military outpost is a natural setting for a handful of clearly countable, standard-complexity structures (a watchtower, a perimeter fence, a jeep, a bunker, a radar installation, barracks, a crate stack, a tank, and a gun emplacement) that will, in a later phase of the project, be watched over by considerably more independent light sources than there are objects – floodlights, fence lamps, headlights, and window glow – which is exactly the object-to-light ratio the course's illumination requirements call for. Independently of illumination, the same nine objects already give this report's milestone a rich, varied demonstration of procedural geometry and, in three cases, multi-level transform hierarchies.

1.1 Objectives

This stage of the project set out to:

1. Build a minimal, theme-agnostic OpenGL abstraction layer (shader compilation and uniform upload, vertex/index buffers, a vertex array description, and GL error checking) in Odin, with no third-party graphics or math library beyond Odin's own standard `core:math/linalg` package.
2. Implement procedural mesh generation restricted to exactly three base primitives – cube, tetrahedron, and plane – each with vertex positions computed from a runtime size parameter, never

- a literal coordinate list, plus a general instancing/combination mechanism capable of assembling every other required shape from these three.
3. Construct a minimum of six (targeted: nine) standard-complexity scene objects entirely from this procedural system, including at least one deep parent/child hierarchy suitable for demonstrating correct nested-transform composition.
 4. Implement a free-fly camera with both perspective and orthographic projections, switchable at runtime.
 5. Implement two operating modes required by the course – an automatic “loop” mode and an interactive, user-controlled mode – sharing a single rendering code path, differing only in what drives the transform matrices each frame.
 6. Verify every claim above with automated tests and reproducible, headless screenshot evidence, rather than relying on visual inspection alone.

1.2 Scope of this report

This is an interim *progress* report, not a final project report. It documents the state of the renderer as of the current milestone – procedural geometry, the transform hierarchy, the camera system, and both operating modes – and is deliberately scoped to that milestone. Illumination models, multi-light support, shading-method comparison, back-face culling and hidden-surface-removal demonstrations, and the ray-tracing/path-tracing/photon-mapping components of the syllabus are addressed in Section 12 as planned work for the next development phase, and are not claimed as complete here.

2 Technology Stack and Development Environment

Table 1: Technology stack.

Component	Choice
Language	Odin
Graphics API	OpenGL, core profile (version 3.3)
Shading language	Hand-written GLSL (no third-party shader code)
Windowing / input	GLFW, via Odin’s <code>vendor:glfw</code> bindings
GL function loading	Odin’s <code>vendor:OpenGL</code> loader
Vector / matrix math	Odin’s own <code>core:math/linalg</code>
Build system	The Odin compiler’s own package-based build (<code>odin build/odin test</code>)

No game engine, scene-graph library, or third-party math library is used anywhere in the project. `core:math/linalg` supplies vector and matrix *types* and standard operations (translation, rotation, projection matrices, and so on) – everything object-specific, including the entire scene hierarchy, is hand-written. No textures or imported 3D models are used at any point; every colour in the current build is a flat, per-object material value.

3 Design Constraints and Principles

The project is governed by a small number of hard constraints, stated here because they directly shape the architecture described in the remaining sections:

- **No precomputed data.** Vertex positions, object placements, camera paths, and animation values are never stored as literal arrays of numbers. Everything is produced by a runtime formula – a

loop, a trigonometric function, a spacing rule – evaluated as needed. Small *parameters* that feed such a formula (a radius, a count, a height, a colour) are permitted; literal position or vertex *tables* are not.

- **Only three base primitives.** The only mesh shapes generated directly are a cube (general box), a tetrahedron, and a plane, each with procedurally computed vertices. Every other shape – a cylinder-like leg, a cone-like roof, a wedge, a composite object – is assembled at runtime by combining multiple instances of these three via matrix transforms, never by a dedicated shape-specific vertex generator.
- **No scene-graph library.** The transform hierarchy described in Section 7 is a small, hand-rolled parent/local-transform system, not an imported dependency.
- **No imported assets.** All geometry is procedural; no 3D model files or textures are loaded.
- **A single render path for both operating modes.** The two modes described in Section 9 differ only in what *drives* the transform matrices each frame (a scripted formula versus live user input) – never in how a frame is actually drawn.

4 System Architecture

The codebase is organized into independent Odin packages with clear responsibilities, rather than one flat collection of source files. Figure 1 shows the dependency structure.

- **Engine** – a theme-agnostic OpenGL wrapper (shader compilation and typed uniform upload, vertex/index buffers, a vertex array/layout description, a minimal renderer, and GL error checking). Knows nothing about the FOB theme.
- **Geometry** – theme-agnostic procedural mesh generation, restricted to the three base primitives from Section 3, plus the mesh-combination primitive every composite shape is built from.
- **Camera** – a free-fly camera, both projection matrices, and the (theme-agnostic) automatic patrol-path mathematics, with no GLFW or windowing dependency.
- **Scene** – the SENTINEL-specific transform hierarchy and the procedural construction of all nine scene objects, built on top of `Geometry`.
- **Source (application)** – the entry point: window and GL context creation, the main render loop, input handling, and the logic that switches between the two operating modes.

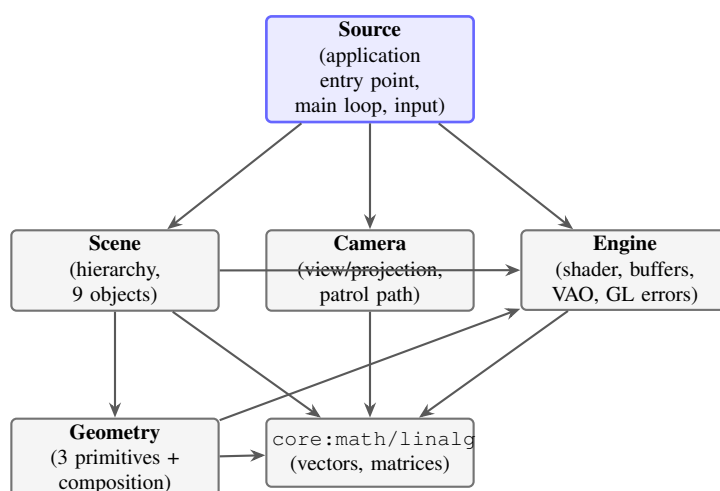


Figure 1: Package dependency structure. `Engine` and `Geometry` are theme-agnostic and know nothing about SENTINEL’s own objects; `Scene` and `Camera` sit above them, and the `Source` application ties everything together.

The `Engine` layer began from a smaller, earlier learning-project skeleton and was audited and con-

firmed compatible with this project's constraints rather than rewritten outright: its uniform-upload procedures already operate directly on `core:math/linalg` matrix types.

5 Procedural Geometry Generation

5.1 Base primitives

Three primitives are generated directly, each parametrized entirely by a runtime size argument:

- **Cube** – full width/height/depth given as parameters; loops over its six faces, computing each face's four corners from a fixed sign-combination pattern scaled by the given half-extents. 24 vertices (four per face, for flat per-face normals), 12 triangles.
- **Tetrahedron** – four corners at alternating corners of a cube of the given size (the standard regular-tetrahedron-inside-a-cube parametrization), scaled by a runtime size argument. 12 vertices, 4 triangles.
- **Plane** – a single quad of the given width/height in the local XY plane, facing $+Z$. 4 vertices, 2 triangles.

Every generator computes a geometric normal from the winding of the vertices it produces and automatically orients that winding so the stored normal points outward from the primitive's own centre – so no face's winding is ever hand-derived and hardcoded.

5.2 Composition via runtime transforms

A single mesh-combination primitive copies one mesh's vertices and indices into another, applying a transform matrix to each copied position (and the corresponding inverse-transpose normal matrix to each copied normal, so normals remain correct under non-uniform scale). Every shape beyond the three base primitives – a cylinder-like tower leg, a radar dish, a wheel rim, a sandbag ring, a cone-like roof – is built by calling this combination primitive repeatedly at runtime, most commonly in one of two patterns:

- **Ring composition.** N copies of a small box (or plane) segment are placed around a circle by a runtime loop that computes a rotation matrix per segment – a low-poly approximation of a cylinder, cone, or ring, with the segment count itself a small, freely chosen parameter (this project uses 6–10 segments per ring, consistent with its low-poly visual style).
- **Angled panel composition.** A single box, scaled thin and rotated by an angle solved from two geometric reference points (for example, a roof's rise and run), used for sloped surfaces such as roof slabs and glaciis plates.

No shape in the scene has its vertex data written out as a literal coordinate list, and no composite mesh is built once and cached; each object's geometry is (re)constructed by these same runtime formulas every time the program starts.

6 Scene Composition

Nine objects are currently built, exceeding the minimum requirement of six. Table 2 lists each object, its child nodes (if any), and the composition technique used to build it.

Table 2: The nine SENTINEL scene objects.

Object	Child nodes	Notes
Watchtower	Floodlight head	Four legs, platform, railing ring, and a four-slab pyramid roof; the head is a separate child node so it can rotate independently.
Perimeter fence & gate	Gate	A full rectangular loop of posts and panels generated by walking the fence boundary with a parametric formula, leaving a gap for the gate.
Armored jeep	Windshield, left/right headlight	Body, cabin, hood, and four ring-composed wheels.
Sandbag bunker	—	A U-shaped wall of two stacked, staggered rows of ring-composed “sandbags.”
Radar mast	Dish, beacon	A tilted, ring-composed dish and a beacon, each a separate child for independent animation.
Barracks hut	Three windows	Box body with a two-slab wedge roof; the windows are child nodes on the rear wall.
Cargo crate stack	—	Six crates, each with a size/rotation/offset determined by a deterministic function of its position in the stack.
Battle tank	Turret → Barrel, Periscope	The project’s deepest hierarchy: hull (root) → turret (child) → barrel and periscope (grandchildren of the hull).
Gun emplacement	—	A sandbag ring, a three-leg tripod, and a gun body; deliberately static, with no children.

Figures 2 and 3 show the current renderer output. Every object is shown with a single flat colour and no lighting model applied – illumination is explicitly out of scope for this milestone (Section 12), so what is visible here is exactly the procedural geometry and the transform hierarchy this report documents, with nothing added on top.

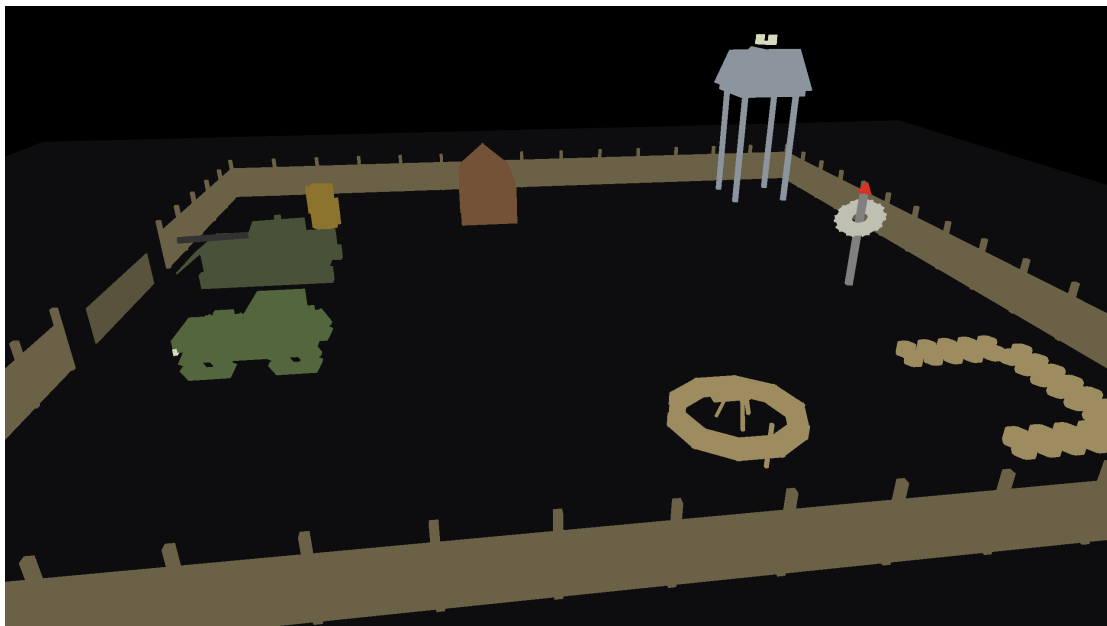


Figure 2: Patrol Mode: the automatic, formula-driven camera circling the base. Every object is procedurally generated; colours are flat per-object materials with no illumination applied at this stage.



Figure 3: Inspection Mode's default free-fly view of the same scene.

7 Transform Hierarchy System

Rather than an imported scene-graph library, SENTINEL uses a small, flat array of nodes, each storing a name, an optional parent index, a local transform (position, Euler rotation, scale), and its own mesh and material. Using a flat array with integer parent indices – rather than a tree of pointers – means a node's index remains valid across the underlying array's own growth as more nodes are added.

Every node's world matrix is computed by a single top-down pass over the array: a root node's world matrix is simply its own local matrix; any other node's world matrix is its parent's (already-computed) world matrix multiplied by its own local matrix. Nodes are guaranteed to be added to the array only after their parent already exists, so this single pass is always sufficient – no second pass or explicit topological sort is required. This computation is performed fresh every frame, never cached, so that any change to a node's local transform (whether from an automatic animation or an interactive edit) is correctly reflected the very next frame it is drawn, and correctly propagates to every descendant.

This mechanism is exercised most thoroughly by the battle tank's hull → turret → {barrel, periscope} chain (Table 2): rotating the hull carries the turret (and everything mounted on it) with it, while subsequently rotating the turret independently moves only the turret and its own children, without disturbing the hull. Correct composition through this kind of nested chain is verified directly by automated tests (Section 10).

8 Camera System and Projections

A single camera type supports both a free-fly mode (position, yaw, and pitch, driven by keyboard and mouse input) and a formula-driven automatic path, and can build either a perspective or an orthographic projection matrix from the same state. Both projection matrices are constructed directly from Odin's `core:math/linalg` package, and both are switchable live, at runtime, without interrupting rendering. When switching from perspective to orthographic, the orthographic view volume's size is derived from the camera's current distance to a fixed scene-focus point, so the visible framing does not jump at the moment of the switch.

The automatic patrol path is itself a pure formula: the camera's position moves along a circle of fixed

radius around the base (with a small sinusoidal height bob), while its look direction is solved, every frame, to aim at a slowly drifting target point near the base's centre – so, consistent with Section 3, no camera waypoint is ever stored as data; the entire path is recomputed from the current animation time on every frame.

9 Operating Modes

The course requires two runtime-toggable operating modes: an automatic loop mode and an interactive, user-controlled mode. Both are implemented and share the exact same drawing code path (Section 3); they differ only in what supplies the values written into each frame's transform matrices.

9.1 Patrol Mode (automatic)

With zero user input, the camera follows the automatic path described in Section 8 while several scene nodes are animated by their own independent, pure time-driven formulas: the watchtower's floodlight head sweeps back and forth, the radar dish spins continuously, the tank turret slowly scans, and the jeep's headlight nodes dip periodically. Every one of these is a rotation of an existing node's local transform, computed fresh each frame from a shared animation clock – never a stored keyframe sequence.

9.2 Inspection Mode (interactive)

Inspection Mode provides a free-fly camera (keyboard movement and mouse look) plus object manipulation:

- **Selection.** Twelve nodes are selectable – the nine root objects plus three notable child nodes (the tank turret, the watchtower's floodlight head, and the radar dish) chosen specifically to demonstrate independent child-transform editing. Selection can step through this list via the keyboard, or a node can be picked directly: a ray is unprojected from the centre of the screen through the inverse of the current view-projection matrix, and tested against each selectable node's own axis-aligned bounding box (computed once from that node's local geometry, then transformed into the node's current world space), with the nearest intersection winning.
- **Translate and rotate.** The selected node's *local* transform can be edited live; because children are always recomposed from their parent's current world matrix (Section 7), editing a parent's local transform is sufficient for every descendant to follow automatically, with no additional propagation step.
- **Reset.** The selected object, or every object at once, can be restored to the transform it had when the program started.
- **Projection toggle.** Perspective and orthographic projection can be switched live in either mode.

Switching between modes does not desynchronize state: leaving Inspection Mode for Patrol Mode resumes the automatic camera path from the nearest point on its circular route to wherever the free camera currently is, rather than jumping to an unrelated point on the path.

10 Verification and Testing Methodology

Because this project has no dedicated graphical debugger and the renderer's own window cannot always be watched interactively during development, two complementary verification mechanisms are used throughout:

- **Automated unit tests.** Odin's built-in testing framework (`odin test`) is used across every package. As of this report, **49 tests pass**, covering: camera and projection mathematics (includ-

ing empirical checks of coordinate-handedness and rotation-direction conventions); procedural-geometry correctness (each generator’s triangle winding matches its own stored normals, normals point outward from the primitive’s centre, and vertex/index counts are exact); the transform hierarchy (including a dedicated multi-level test confirming that rotating an intermediate node correctly carries its children through the resulting composed transform); and the mouse-picking mathematics used by Inspection Mode (ray/box intersection, screen-to-world ray construction, and nearest-hit selection).

- **Headless, reproducible screenshot capture.** The application supports a command-line flag that renders a fixed number of frames, saves the resulting framebuffer to an image file, and exits – allowing the renderer’s actual output to be inspected and compared frame-for-frame without a live, interactively-watched session. This mechanism produced Figures 2 and 3.

11 Current Progress Status

Table 3 summarizes progress against the full course syllabus. Topics in the upper block are complete and demonstrated by this report; topics in the lower block are planned for the next phase of development (Section 12).

Table 3: Progress against the course syllabus.

Syllabus topic	Where it lives	Status
Display/input devices, graphics pipeline	GLFW window and input; a custom OpenGL pipeline	Complete
Scan conversion, polygon filling	GPU rasterization of correctly wound, procedurally generated triangle meshes	Complete
3D primitives, basic & composite transformations	Procedural primitive generators; hierarchical transform chains (Sections 5–7)	Complete
Classical / computer viewing, projective transformation	Free-fly camera; perspective and orthographic projections, toggled live (Section 8)	Complete
Local / global illumination, polygon shading methods	—	Planned
Light types (point, spot, area, directional)	—	Planned
Back-face culling	—	Planned
Hidden surface removal	—	Planned
Ray tracing	—	Planned
Path tracing	—	Planned
Photon mapping [†]	—	Planned

[†]Planned as a written explanation of how the technique would extend the ray-traced component, not a live implementation (Section 12).

Beyond the syllabus mapping, current functional status is: nine procedurally generated objects (target met, minimum of six exceeded); one three-level transform hierarchy (tank hull → turret → barrel/periscope) plus several two-level hierarchies, all verified by automated tests; both required operating modes implemented and sharing a single render path; both projection modes switchable at runtime; and 49 automated tests passing with no known failures.

12 Remaining Work

The following syllabus areas are planned for the next phase of development, in roughly the order they will be undertaken:

1. **Multi-light illumination.** A Blinn-Phong local illumination model (ambient, diffuse, specular, and emission terms) evaluated over multiple simultaneously active lights, with the total active light count planned to exceed the nine-object count by a meaningful margin.
2. **Light types.** Point, spot, area, and directional lights, attached through the existing transform hierarchy so that a light parented to a moving or rotating object (for example, the tank turret's searchlight, or the jeep's headlights) continues to aim and follow correctly.
3. **Shading-method comparison.** Flat, Gouraud, and Phong shading, switchable live for direct visual comparison.
4. **Back-face culling and hidden-surface removal.** A manual, normal-based culling test compared directly against hardware culling, and the GPU depth buffer's role in hidden-surface removal, both made toggleable so the underlying mechanism can be demonstrated rather than only its effect.
5. **Ray tracing.** A small, genuinely ray-traced element (analytic ray/primitive intersection against a compact proxy representation of the scene), rather than a purely decorative approximation.
6. **Path tracing.** A bounded, multi-sample numerical integration technique for at least one area-like light source.
7. **Photon mapping.** Addressed as a written explanation of how the technique would extend the ray-traced component above – photon emission, spatial storage, and a gather pass – rather than implemented directly, since it is fundamentally a batch/ offline technique in tension with this project's own every-frame, nothing-cached design principle (Section 3).
8. **Final documentation.** A complete build/run guide, control reference, and a final report covering the full, completed system.

13 Conclusion

At this stage, SENTINEL has a working, procedurally generated scene of nine objects assembled entirely from three base primitives, a correct and independently tested hierarchical transform system capable of multi-level nested articulation, a camera supporting both standard projections, and two fully functional operating modes – one automatic and formula-driven, one interactive – sharing a single rendering code path, exactly as the course requires. No geometry, placement, or animation data is hardcoded anywhere in the project; everything shown in Figures 2 and 3 is produced by runtime code. The remaining syllabus topics – illumination, shading, culling and hidden-surface-removal demonstrations, and the ray/path/photon-tracing components – are scoped and planned (Section 12) as the next phase of this project's development.

References

- [1] Odin Programming Language, Official Documentation. <https://odin-lang.org/docs/>
- [2] Khronos Group. *OpenGL 3.3 Core Profile Specification*. <https://www.khronos.org/registry/OpenGL/specs/gl/glspec33.core.pdf>
- [3] GLFW: An OpenGL Library for Creating Windows, Contexts, and Surfaces. <https://www.glfw.org/documentation.html>